

and (2) a Docker Engine, which runs the Docker containers.

The architecture is well supported by the other ecosystem players like Git, Jenkin, Maven, etc.

As part of the development cycle, the engineers write and push the code to Git repositories. As part of the build cycle, build tools like Jenkins pull the code from Git repositories, build the Docker image and push it to the Docker Hub.

As part of the deployment cycle, the Docker runtime pulls the image from Docker Hub and creates Docker containers that bring the applications to life. Any number of images can be pulled and launched on a single instance of Docker runtime. Where is the magic?

Docker containers are language-agnostic. They are capable of running applications on any platform. For example, they can run an Express Web service on Node or a Flask Web service on Python, or a Spring Boot Web service on JRE. The answer is simple. The container just offers a sandbox with a Linux environment. The container image must package everything else that is required for the application. For example, the image of a Flask service must include Python runtime into the container image and similarly, the image of a Spring Boot service must include JRE.

Let's explore the last option a bit more.

Spring Boot and Docker container

The Maven build tool packages a Java application. The Docker build tool packages a container. Just like the *pom.xml* acts as the manifest file for the Maven build tool, *Dockerfile* acts as the manifest file for the Docker build tool.

A *Dockerfile* specifies the root container as the prerequisite, and presents directives to be followed in order to create the image. For instance, the following is a manifest for building a Docker image of a Spring Boot Web application.

```
FROM maven:3.5-jdk-8
COPY src /usr/clarimy/src
COPY pom.xml /usr/clarimy
RUN mvn -f /usr/clarimy/pom.xml clean package
EXPOSE 8080
ENTRYPOINT ["java","-jar","/usr/clarimy/target/ums-add-
service.jar"]
```

Let's decipher it line by line.

```
FROM maven:3.5-jdk-8
```

The above directive specifies *maven:3.5-jdk-8* as the root container. This makes sure that a Docker container is started with a JDK of version 8 and the Maven of version 3.5 is available for the rest of the packaging process. This container is used only for building the image. Let's call this

a build-container.

```
COPY src /usr/clarimy/src
COPY pom.xml /usr/clarimy
```

The above directives create */usr/clarimy* folder on the build-container and copy the code from a local *src* folder to a */usr/clarimy/src* folder on the build-container. They also copy the local *pom.xml* file to */usr/clarimy* on the build-container.

```
RUN mvn -f /usr/clarimy/pom.xml clean package
```

This directive kicks off the Maven build process to package the Java code as per the referred *pom.xml*. Note that all this happens on the build-container. Once the process is completed, the container image for the Spring Boot application is ready. This image can be run as a container, which we refer to as deployment-container. However, we still need to specify the directives on how to launch this on the deployment-container.

```
EXPOSE 8080
```

The above directive specifies that port 8080 be opened up for traffic on the deployment-container when this image runs.

```
ENTRYPOINT ["java","-jar","/usr/clarimy/target/ums-add-
service.jar"]
```

This directive specifies the command to launch the application on the deployment-container.

There are several other directives possible, but the approach is more or less similar. Once the *Dockerfile* is ready, and the Docker is installed on a given machine, the following command on a Docker runtime is sufficient to build the container image.

```
docker build -t clarimy/ums-add-service .
```

Observe that the last token of the above command points to the location of *Dockerfile*, which is the current folder in this case.

The following command lists all available images on the local Docker Engine:

```
docker images
```

Figure 3 presents a sample output of the above command. This list includes the images that are pulled from the Docker Hub as well as built locally.

It's not useful if the created Docker image of the application just lies on the local Docker Engine. In order